

모든 프로젝트에 공통된 흐름 — 원본을 확인하고, 다시 만들 수 있게 적재하고, 다른 방법으로 한 번 더 검증한다.

PORTFOLIO 2026

데이터를 믿을 수 있게 만드는 과정

원본 검증 · 재현 가능한 파이프라인 · 독립 검증

전종현

PROJECT PRINCIPLES

프로젝트 공통 원칙

1 · 원본부터 확인

공식 제공처의 출처·라이선스를 확인하고, 내려받은 파일을 MD5/SHA-256으로 대조합니다.

2 · 다시 만들 수 있게

명령 몇 줄로 원본 → DB → 보고서를 다시 생성합니다. Windows와 Ubuntu(WSL2)에서 실행을 확인했습니다.

3 · 다른 방법으로 한 번 더

Python 집계 결과를 별도 SQL로 다시 계산해 대조합니다. 경계 사례는 자동 테스트로 남깁니다.

4 · 해석의 경계를 적기

가정으로 계산한 값을 성과로 부르지 않습니다. 모의 데이터와 실제 데이터를 구분해 적습니다.

작업 방식을 먼저 밝힙니다

아래 프로젝트는 **AI 코딩 에이전트(Codex)로 구현**했습니다. 제가 한 일과 에이전트가 한 일은 다음 장에 나눠 적었습니다.

CONTENTS

01 **구매 로그 159만 건 파이프라인** 06
— 공개 업무 로그를 SQLite로 적재하고 병목 구간을 독립 검증

02 **EV Factory Lab** 10
— 데이터 계약 · CSV→SQLite · 시간순 분할 · 공정 시뮬레이션

03 **PCB 결함 검출 평가** 12
— 그룹 단위 분할로 데이터 누수를 막은 비전 모델 평가

04 **제조 이상 사건 저장·조회** 14
— 센서 사건을 SQLite에 중복 없이 적재하고 HTTP API로 제공

05 **DX 과제 우선순위 Copilot** 15
— 감사 이력 해시 연결 · 재시도 안전성(Idempotency-Key)

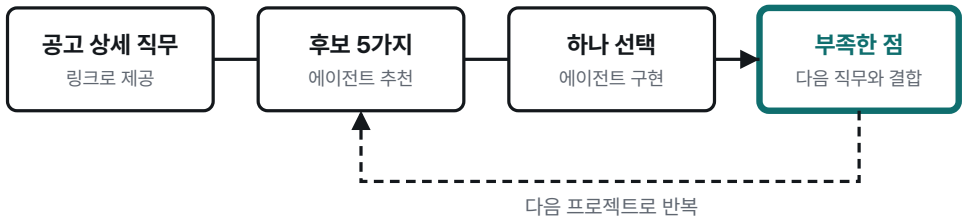
06 **기초 재구축** 16
— C++ 기초부터 다시 · 설비 텔레메트리 87만 행 처리 엔진 계획

HOW I BUILT THIS

에이전트와 일한 방식

무엇을 만들지는 제가 정하고, 구현은 에이전트(Codex)에게 맡겼습니다

포스텍에서 열린 LG DAY 채용설명회에서 많은 대기업이 AX 직무를 새로 만들어 신입사원을 뽑는다는 이야기를 들었습니다. 그래서 채용 공고의 상세 직무를 링크로 Codex에 주고, 그 업무에서 해볼 만한 1인 프로젝트 5가지를 추천받아 하나를 골라 완성하게 했습니다. 첫 프로젝트가 PCB 결함 검출(심텍 직무)이었고, 이후에는 앞 프로젝트에서 부족했던 부분과 새 공고의 상세 업무를 합쳐 다음 프로젝트를 만드는 과정을 반복했습니다.



제가 한 일	에이전트가 한 일
AX 직무가 새로 생기는 흐름을 보고 방향 결정	공고에 맞는 프로젝트 후보 제안
공고의 상세 직무를 근거로 제공	공개 데이터 조사와 선정
추천 후보 중 만들 것을 선택	코드, 테스트, 화면, 문서 구현
앞 프로젝트의 부족한 점을 보고 다음 방향 지시	검증 실행과 결과 기록

TURNING POINT

맡기기만 하면 남는 것이 없었습니다

Jetson 핀이 열려 있지 않은 문제를 겪으면서, 에이전트에게 모든 것을 맡기면 제가 무엇을 했는지 모르는 상황, 즉 남는 것이 없는 상황이라는 것을 느꼈습니다.

그래서 방식을 바꿨습니다

지금은 에이전트에게 완성품 대신 **요구사항, 생각하는 순서, 힌트**만 받고 코드는 제가 직접 씁니다.

- **SWEA**(삼성 SW Expert Academy)로 C++ 기초부터 코딩 공부
- **Ollama**로 로컬 LLM을 한 단계씩 직접 설치하고 연결
- 기존 1인 프로젝트의 핵심 로직을 직접 다시 구현

여기서 배운 것

에이전트는 만들고 싶은 결과를 분명하게 정의할 수 있는 사람에게 가장 강한 도구입니다. 저는 결과를 정의하고 방향을 정하는 일을 해 왔고, 이제 그 결과를 이해하고 직접 고칠 수 있는 실력을 쌓고 있습니다.

PROJECT 01

구매 로그 159만 건, 어느 구간이 오래 걸리나

공개 업무 로그(XES 729 MB)를 스트리밍 변환해 SQLite에 적재하고, 구간별 경과시간을 집계·독립 검증

유형 1인 프로젝트 **데이터** BPI Challenge 2019 (4TU.ResearchData, CC BY 4.0)
스택 Python 표준 라이브러리 · SQLite · Node 테스트

실제 기업의 익명화된 구매 기록 **1,595,923개 이벤트**를 대상으로, 처리 흐름에서 시간이 가장 오래 쌓이는 구간을 찾았습니다. 공식 메타데이터 API로 원본 위치와 MD5를 확인한 뒤 내려받고, 외부 패키지 없이 스트리밍으로 변환했습니다. 비교 조건과 제외 사유는 사례마다 남겨, 숫자 하나하나를 원본 이벤트까지 추적할 수 있게 했습니다.

ETL · SQLite · Process Mining · Data Validation

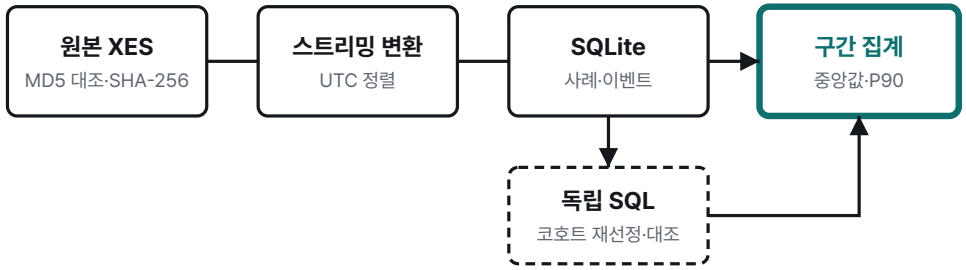


Fig 1. 데이터 흐름 — 같은 결과를 Python과 SQL 두 경로로 계산해 대조

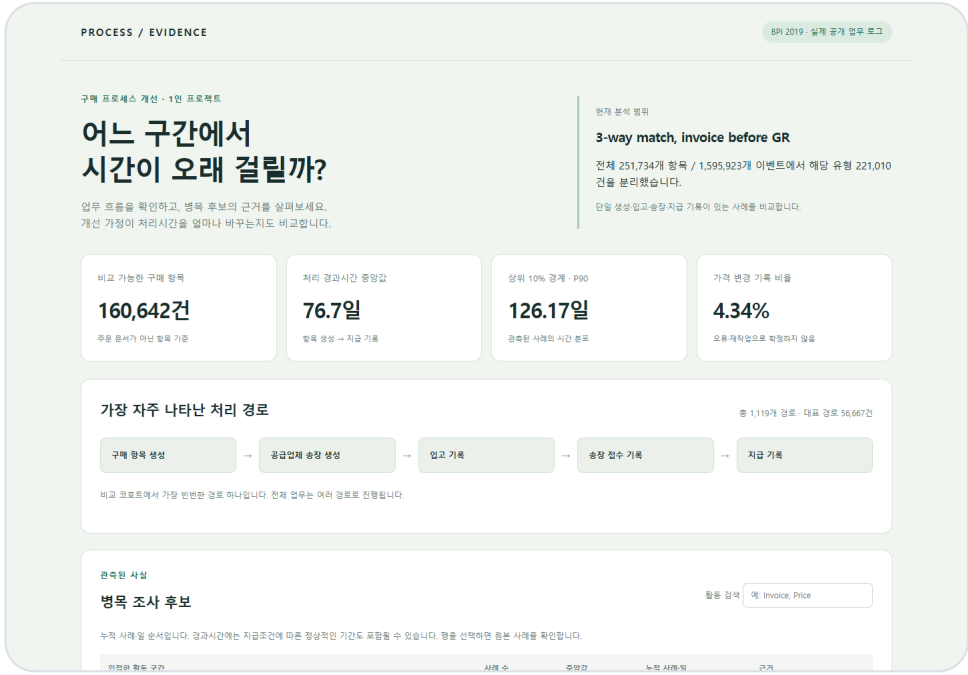


Fig 2. 결과 화면 — 비교 대상 160,642건 · 중앙값 76.7일 · P90 126.17일 · 1,119개 처리 경로 중 대표 경로

로컬 읽기 전용 서버(Python 표준 라이브러리)로 제공. 활동 검색과 사례 타임라인을 지원합니다.

01 — 비교 대상을 어떻게 골랐나

모든 구매를 한꺼번에 비교하지 않은 이유

입고가 필요한 구매, 입고 없이 처리하는 구매, 위탁 구매는 기대하는 흐름이 다릅니다. 한 유형(3-way match, invoice before GR)을 고르고, 생성·입고·송장·지급이 각각 한 번인 사례만 비교했습니다. 제외는 "데이터가 잘못됐다"는 판정이 아니라 비교 범위를 좁히는 규칙입니다.

원본 구매 항목	**251,734**
선택 유형	**221,010**
비교 조건 충족	**160,642**
제외 60,368	

제외 사유: 취소·삭제, 시간 순서 모순, 동일 활동·동일 시각 중복 후보, 반복 입고·송장·지급

Fig 3. 코호트 선정 — 모든 사례의 포함 여부와 제외 사유를 JSON으로 보존

76.70일
생성 → 지급 기록
중앙값

126.17일
P90
(상위 10% 경계)

4.34%
가격 변경 기록이 있는
사례 6,970건

01 — 검증

숫자를 믿을 수 있게 만든 장치

- Python으로 만든 코호트를 **별도 SQL 조건으로 다시 선정**해 사례 ID 집합을 대조
- 모든 인접 구간 시간의 합이 사례 전체 시간과 일치하는지 확인
- 보고서에 노출한 근거 전이 **990개**를 DB의 이벤트 순번·활동·시간차와 전부 대조
- 누락 지급, 분할 입고, 취소, 시간대, 동시각 이벤트, 잘못된 원본 해시를 자동 테스트 — Python 17개 · 화면 로직 7개
- 실제 브라우저(Chrome) 검사 10개 — 검색, 사례 선택, 시나리오 변경, 오류 복구, 모바일 폭
- Ubuntu 24.04(WSL2)에서 테스트·검증·서버 응답(HTTP 200) 재확인

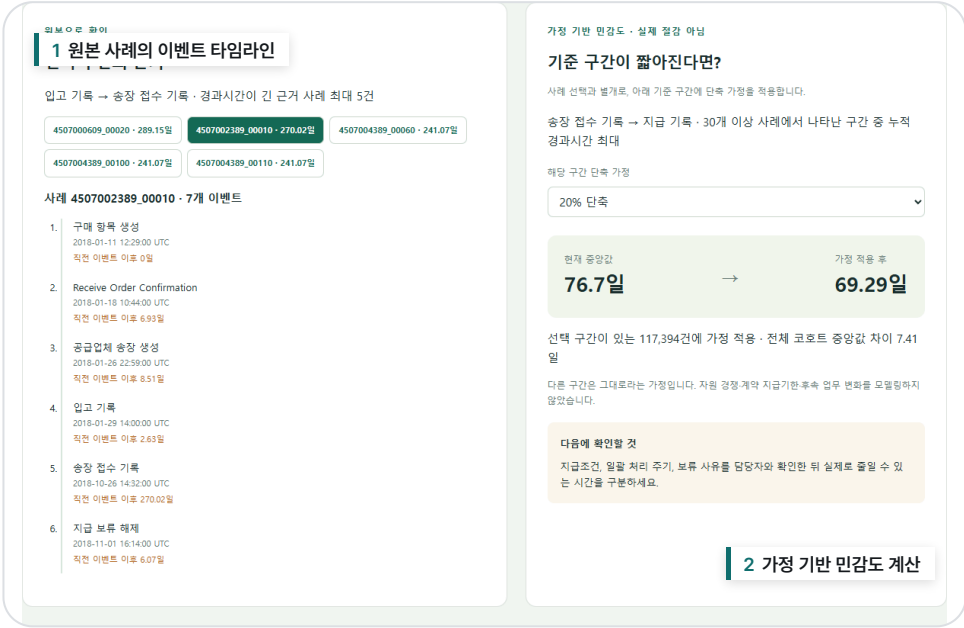


Fig 4. 근거 추적 화면 — 수치를 원본 사례까지 거슬러 확인

해석의 경계. "송장 접수 → 지급" 구간을 20% 줄인다고 가정하면 중앙값이 69.29일(−7.41일)로 계산됩니다. 이것은 **산술적 가정**이며 실제 절감 실적이 아닙니다. 계약상 정상 지급기간이 포함되어 있을 수 있습니다.

PROJECT 02

EV Factory Lab

모의 전동화 부품 조립 라인의 공정 데이터를 데이터 계약으로 검증하고 SQLite 이력으로 적재해, 불량 위험 분석과 병목 실험을 연결

유형 1인 프로젝트 **데이터** 시드 고정 모의 데이터 (실제 공장 데이터 아님)

스택 Python · pandas · scikit-learn · SQLite

원본 CSV를 해시와 함께 남기고, 데이터 계약(컬럼·범위·형식)을 통과한 데이터만 SQLite 제품·공정 이력으로 적재합니다. 같은 DB에서 두 갈래 분석을 합니다. 하나는 **시간순 학습·검증·시험 분할**로 미래 데이터가 학습에 섞이지 않게 한 불량 위험 점수이고, 다른 하나는 검사 셀 증설·공정 편차 축소를 **30쌍 반복 실험**으로 비교한 병목 분석입니다.

Data Contract | CSV→SQLite | Time Split | Simulation

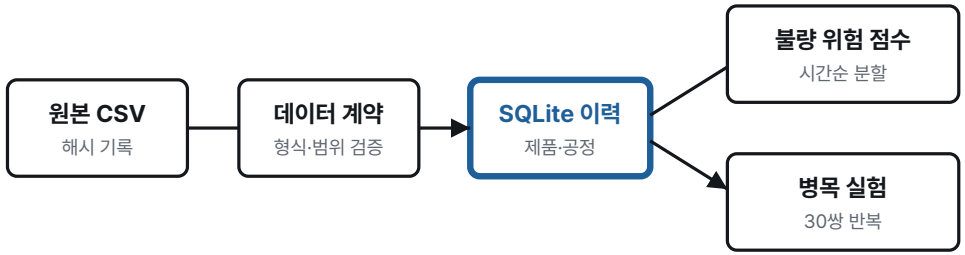


Fig 5. 하나의 DB, 두 갈래 분석

시험 불합격률 약 29%는 결함을 자주 일으킨 스트레스 시나리오 값이며 실제 공장 불량률이 아닙니다. Ubuntu에서 빌드와 테스트 21개 통과를 확인했습니다.

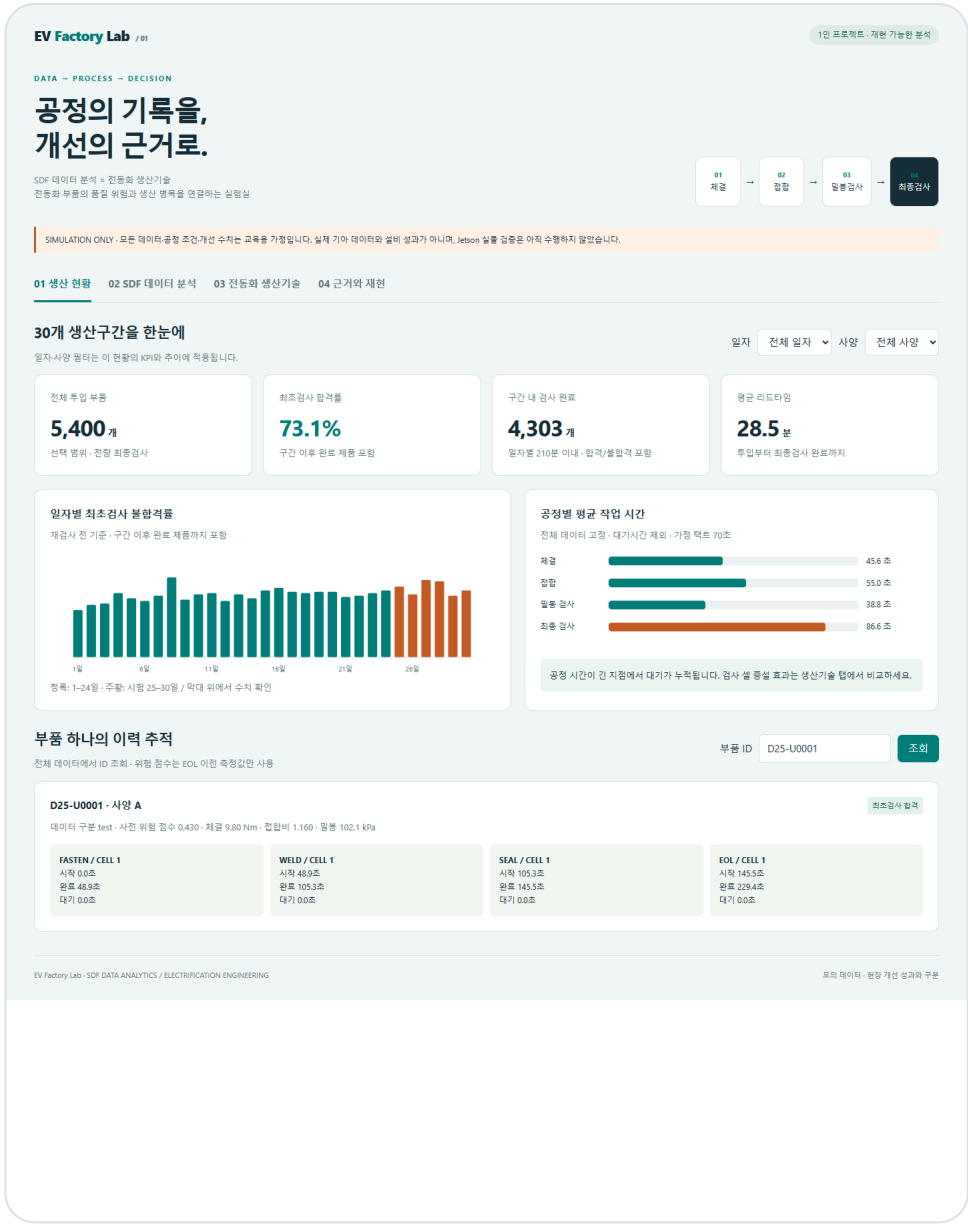


Fig 6. 대시보드 — 투입 5,400개 · 최초검사 합격률 73.1% · 일자별 불합격률(학습/시험 구간) · 부품 ID별 공정 이력 조회

설치·API 키·인터넷 없이 열리는 정적 화면. 상단 배너에 "모의 데이터"임을 표시합니다.

PROJECT 03

PCB 결함 검출 평가

공개 PCB 이미지 1,500쌍을 학습 데이터로 변환하고, 같은 기판 그룹이 학습·시험에 섞이지 않게 나눠 성능을 정직하게 측정

유형 1인 프로젝트 **데이터** DeepPCB 공식 저장소 (SHA-256 기록) **스택** Python · YOLO11n · CPU 학습

이미지 단위로 무작위 분할하면 같은 기판의 비슷한 이미지가 학습과 시험에 동시에 들어가 성능이 부풀려집니다. 그래서 **원천 PCB 그룹 단위로 분할**하고, 시험 그룹은 모델·임계값 선택에 쓰지 않았습니다. 결과에서 좋은 숫자만 고르지 않고 오탐률도 함께 보고했습니다.

보류 시험 그룹 734장 (신뢰도 0.25, IoU 0.5)	값
mAP50 / mAP50–95	**90.8%** / 62.8%
결함 객체 재현율 (2,561개 중 2,329개)	90.9%
정밀도	67.4%
정상 이미지 오탐률 (367장 중 165장)	**45.0%**

그래서 내린 판단. 정상 이미지의 45%에서 오탐이 나므로 이 임계값은 자동 판정에 쓸 수 없습니다. 검증 그룹에서 임계값을 0.25 → 0.75로 올리면 오탐률은 48.6% → 6.8%, 재현율은 81.4% → 53.2%로 바뀝니다. 새 임계값은 새 보류 데이터로 다시 확인해야 합니다.

Ubuntu에서는 데이터 변환·평가 로직 테스트 6개만 확인했고, 학습은 Windows CPU에서만 실행했습니다.

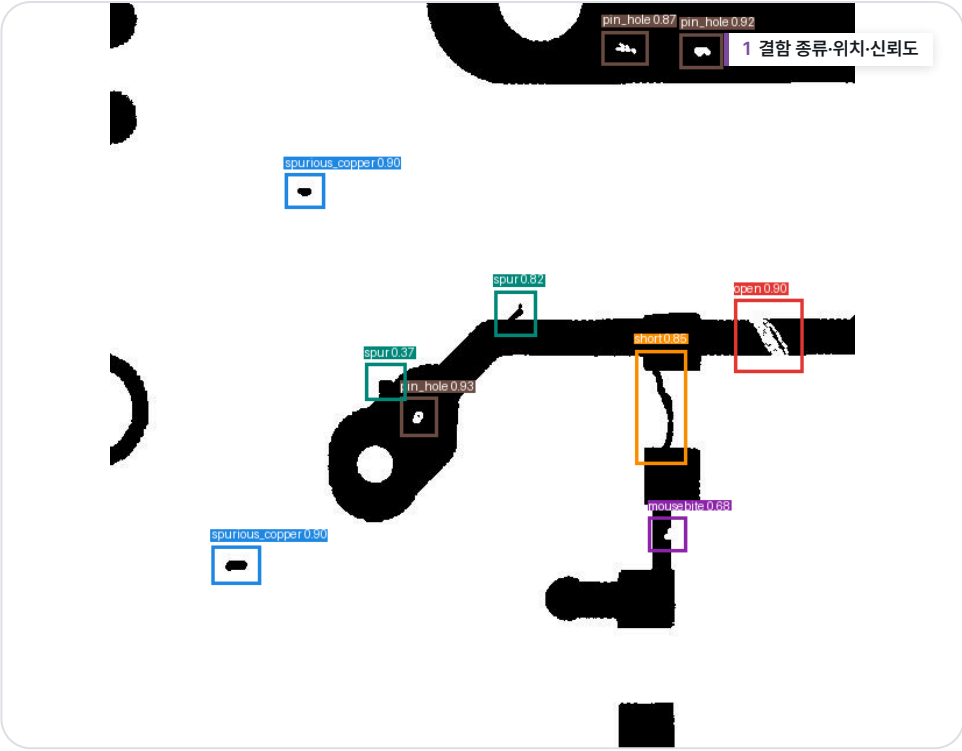


Fig 7. 단일 이미지 검사 결과 — open, short, mousebite, spur, spurious_copper, pin_hole

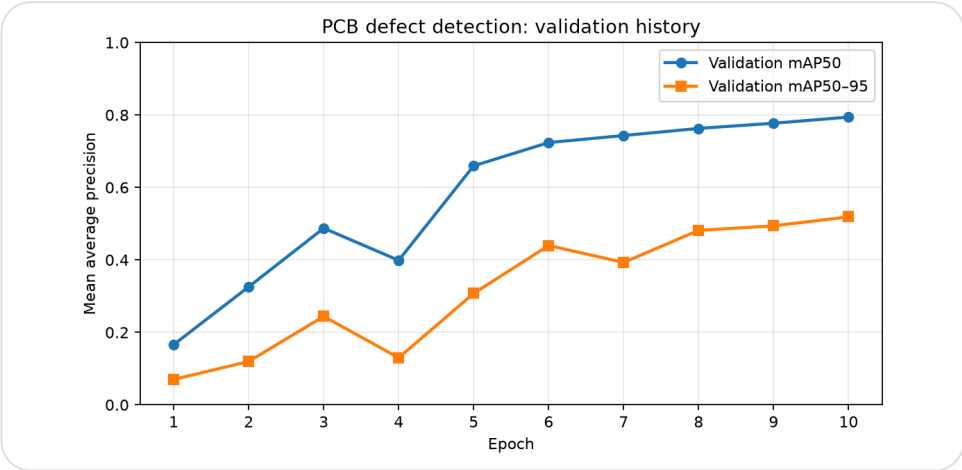


Fig 8. 학습 회차별 검증 mAP — 10 epoch, 최고 가중치 검증 mAP50 79.4%

PROJECT 04

제조 이상 사건 저장·조회

공개 센서 데이터 기준선 → 사건을 SQLite에 중복 없이 적재 → HTTP 읽기 API

유형 1인 프로젝트 **스택** Node.js (외부 의존성 없음) · SQLite



공식 데이터를 내려받을 때 SHA-256을 검사하고, 일치하는 파일이 있으면 다시 받지 않습니다. 같은 데이터·코드·설정이면 생성 시각 외 결과가 같습니다. 사건 import를 반복해도 같은 사건이 두 번 저장되지 않게 했습니다. 다중 운전 모드 데이터(Tennessee Eastman)는 모드별 기준선과 시간 holdout을 따로 검증했습니다.

75
Ubuntu(WSL2)에서 통과한 자동 테스트

5
감사 시나리오 + 근거 ID 위조 응답 검사

로컬 LLM은 설명 보조에만 쓰고, 안전 상태 결정은 규칙 기반 FSM이 합니다. 현장 성능은 검증하지 않았습니다.

PROJECT 05

DX 과제 우선순위 Copilot

과제 후보를 설명 가능한 6개 항목으로 점수화하고, 승인·반려 이력을 위변조 감지 가능한 감사 로그로 남기는 로컬 파일럿

유형 1인 프로젝트 **스택** Node.js · 파일 저장소 · REST API

데이터 엔지니어링과 직접 닿는 부분은 **데이터를 안전하게 쓰고 남기는 방법**입니다.

구현한 장치	왜 필요한가
Idempotency-Key	같은 요청을 다시 보내도 결과를 두 번 저장하지 않음
감사 이벤트 해시 연결	기록이 중간에 바뀌면 감지
입력 경계 검증	빈 값·길이·ID·URL 형식 오류를 저장 전에 차단
기준값 없으면 보류	근거 없는 KPI 목표를 만들지 않음 (`baseline_required`)
역할별 권한	viewer 읽기 / reviewer·admin 쓰기 분리

23
Ubuntu(WSL2)에서 통과한 테스트

5
합성 과제 카드 (출처·검토 지점 포함)

상용 운영 아님. 다중 사용자 운영에는 PostgreSQL·SSO·RBAC 교체가 필요하다고 문서에 명시했습니다.

06 — 지금 하고 있는 일

기초 재구축

AI 도구로 만든 결과물을, 직접 설명하고 다시 짤 수 있는 실력으로 바꾸는 중

앞의 프로젝트들은 AI 코딩 도구로 빠르게 구현했습니다. 결과를 내는 데는 효과적이었지만, 코드 한 줄 한 줄을 스스로 짤 수 있는 것과는 다르다고 판단했습니다. 그래서 2026년 10월부터 **C++ 기초 문법부터 직접** 다시 쌓고 있습니다.

단계	내용	상태
기초 0단계	변수·입출력·조건문·반복문 (홀수 합 프로그램)	완료
SWEA 연습	문제 이해 → 입출력 형식 → 손계산 → 설계 → 코드 → 디버깅 6단계 습관	진행 중
로컬 LLM	Ollama 설치, 모델 실행, 연결을 단계별로 직접 실습	진행 중
설비 데이터 엔진	Azure PdM 공개 데이터(텔레메트리 876,100행)를 C++로 읽고 이벤트 집계	계획

디버깅에서 배운 것

"선언되지 않은 식별자" 오류의 원인은 대부분 선언한 이름과 사용한 이름이 다른 경우였습니다. 빌드가 실패하면 IDE가 이전 실행 파일을 그대로 실행해 예전 결과가 보일 수 있다는 것도 직접 겪고 확인했습니다. 결과가 이상하면 **손으로 계산한 값과 처음 달라지는 지점**부터 찾습니다.

[TODO: 지원 전에 실제 진행 상태로 표를 갱신하세요.]

REPRODUCIBILITY

모든 프로젝트를 Linux에서 다시 실행

데이터 플랫폼은 Linux에서 운영됩니다. Windows에서 만든 프로젝트 5개를 Ubuntu 24.04(WSL2)에서 다시 실행하고, Windows 전용 실행 스크립트(.ps1)에는 같은 일을 하는 .sh를 추가했습니다.

프로젝트	Ubuntu에서 확인한 것
01 구매 로그 파이프라인	테스트 17+7개, 검증, 원본 검사, 서버 HTTP 200
02 EV Factory Lab	빌드, 테스트 21개
03 PCB 결함 검출	로직 테스트 6개 (학습은 미실행)
04 제조 사건 저장·조회	테스트 75개, 데이터 내려받기·SHA-256 일치
05 DX 과제 Copilot	테스트 23개, 데모, 코어 검사

확인하지 않은 것도 적습니다: 브라우저 자동 검사는 Linux에서 실행하지 않았고, 로컬 LLM의 Linux 추론은 확인하지 않았습니다.

감사합니다

Email a01089676836@gmail.com **GitHub** [TODO] **Phone** [TODO]